

marp: true

theme: default

paginate: true

style: |

/* — Base — */

@import url('https://fonts.googleapis.com/css2?

family=Inter:wght@400;600;700;900&family=JetBrains+Mono:wght@400;700&display=swap');

section {

font-family: 'Inter', 'Segoe UI', sans-serif;

font-size: 26px;

background: #0d1117;

color: #e6edf3;

padding: 48px 64px;

}

The setup

The hardware

 2 vCPU · 2 GB RAM · 20 GB SSD

 ~\$12/month (Hetzner CX22)

 Docker Compose (same constraints, same demo)

The WordPress site

 2,500 posts · 500 WooCommerce products

 ~6,000 revisions in DB

 Realistic bloat: transients, orphaned meta

 WooCommerce + Yoast SEO active

The load generator



The journey

Level	Stack	p95 Latency	requests/s	CPU	DB Threads	New layer
0	Apache + mod_php	💣 crash @ ~50 VU	~140 → collapse	100%	20+	—
1	+ Nginx · FPM · OPcache · Redis	133 ms	65	25%	12.5	OPcache · Redis
2	+ FastCGI cache · MariaDB tuning	79 ms	70	18%	3	FastCGI page cache
	+ Cloudflare	~15 ms	origin sees			Edge cache

Key concept: what is a Virtual User?

NOT this:

- ✗ 50 VUs = 50 people who visited today
- ✗ 50 VUs = 50 page loads total
- ✗ 1 VU = 1 unique session

THIS:

- ✓ 1 VU = 1 browser-like loop running **continuously**

```
VU #1: GET / → sleep 1-2s  
       → GET /?p=1 → sleep 1-2s  
       → GET / → repeat forever
```

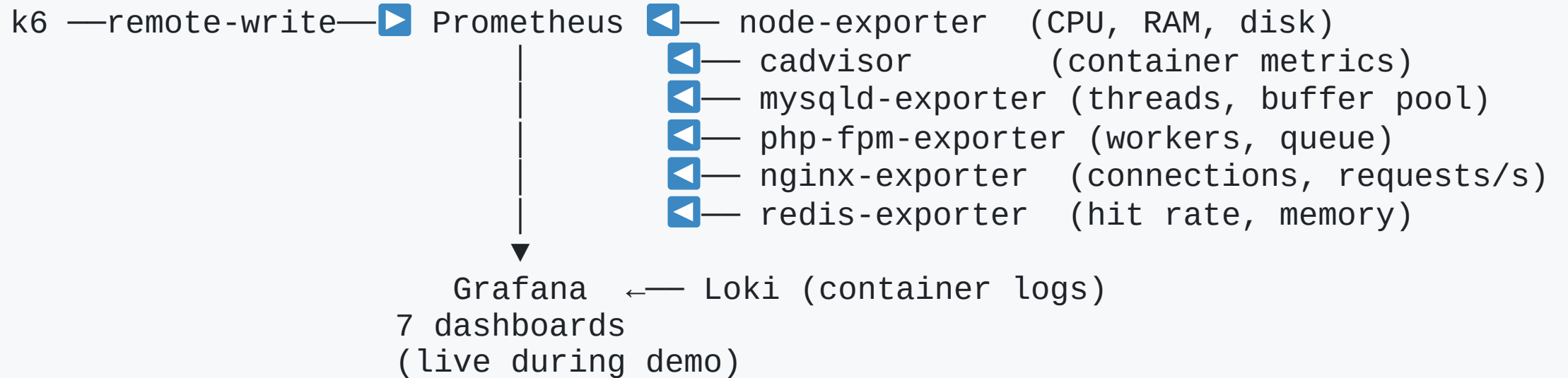
50 VUs = **50 concurrent loops**

= **50 simultaneous open connections**

Key metric definitions

Term	What it actually means
Virtual Users	Concurrent browser-like loops running continuously — not unique visitors
requests/s	Total HTTP responses served per second across all VUs
p95 latency	95% of requests were <i>faster</i> than this — 5% were slower
p99 latency	The near-worst case — 1% of requests were slower than this
FPM workers	PHP processes <i>currently executing code</i> right now
FPM queue depth	Requests waiting for a free PHP worker — 0 = healthy, >0 = trouble
DB threads running	MySQL queries <i>actively executing</i> — not just connected

Observability stack



Every number you see on screen is a **real metric from a real process**.

No synthetic benchmarks. No cherry-picked results.



ACT I

The Crash

Level 0 — Apache + mod_php

Level 0 — How Apache prefork works

Each connection = one OS process

Browser connects

- Apache forks a worker process
- worker loads PHP + WordPress into RAM
- PHP runs, queries MySQL
- response sent
- worker stays alive, waiting (idle, ~32 MB RAM held)

The problem:

- 60 idle workers = 1.9 GB RAM consumed
- Browser #61 gets: queue
- Browser #200 gets: **503**

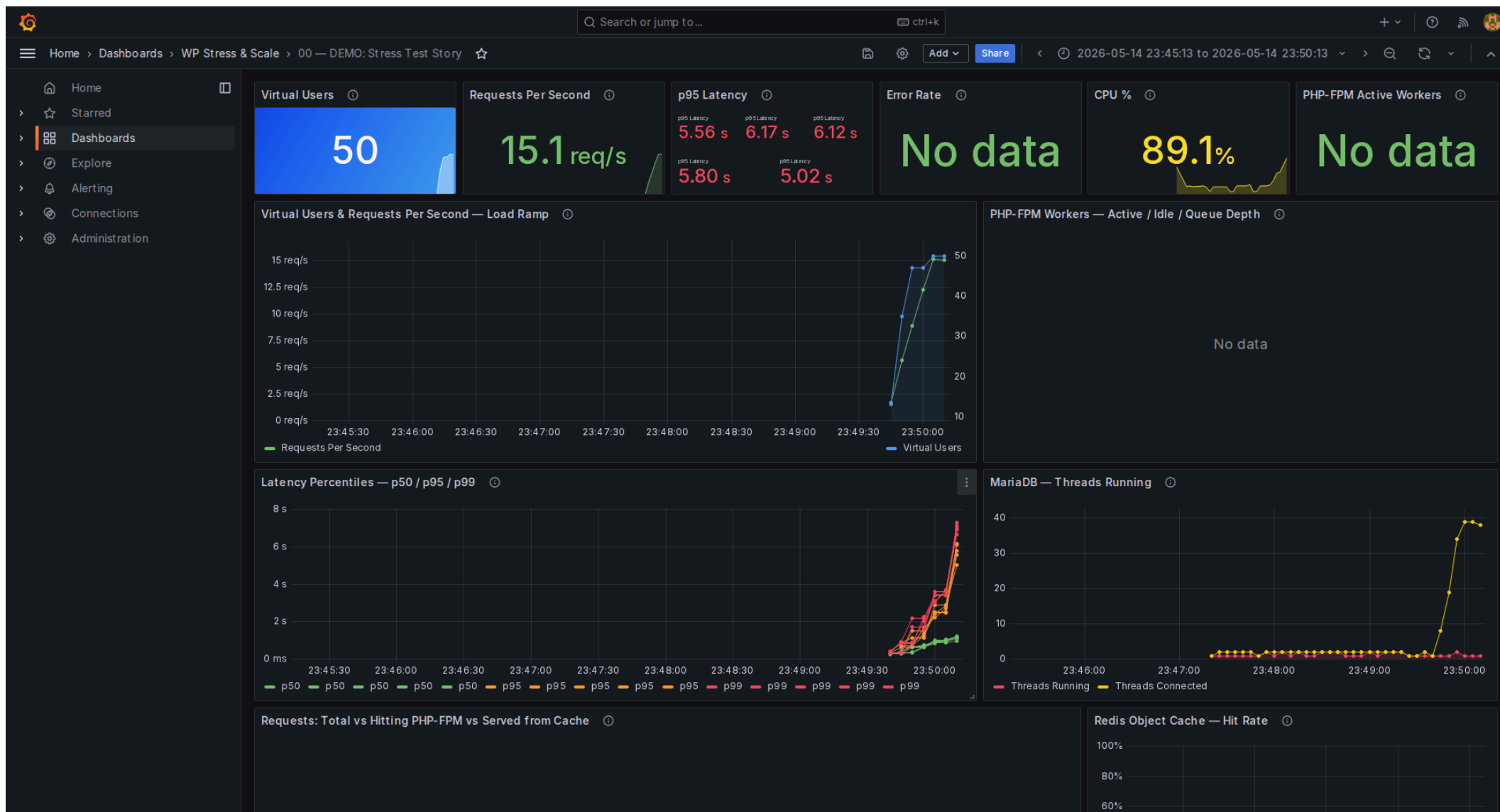
The process model was designed for static files in 1996.

The Apache math

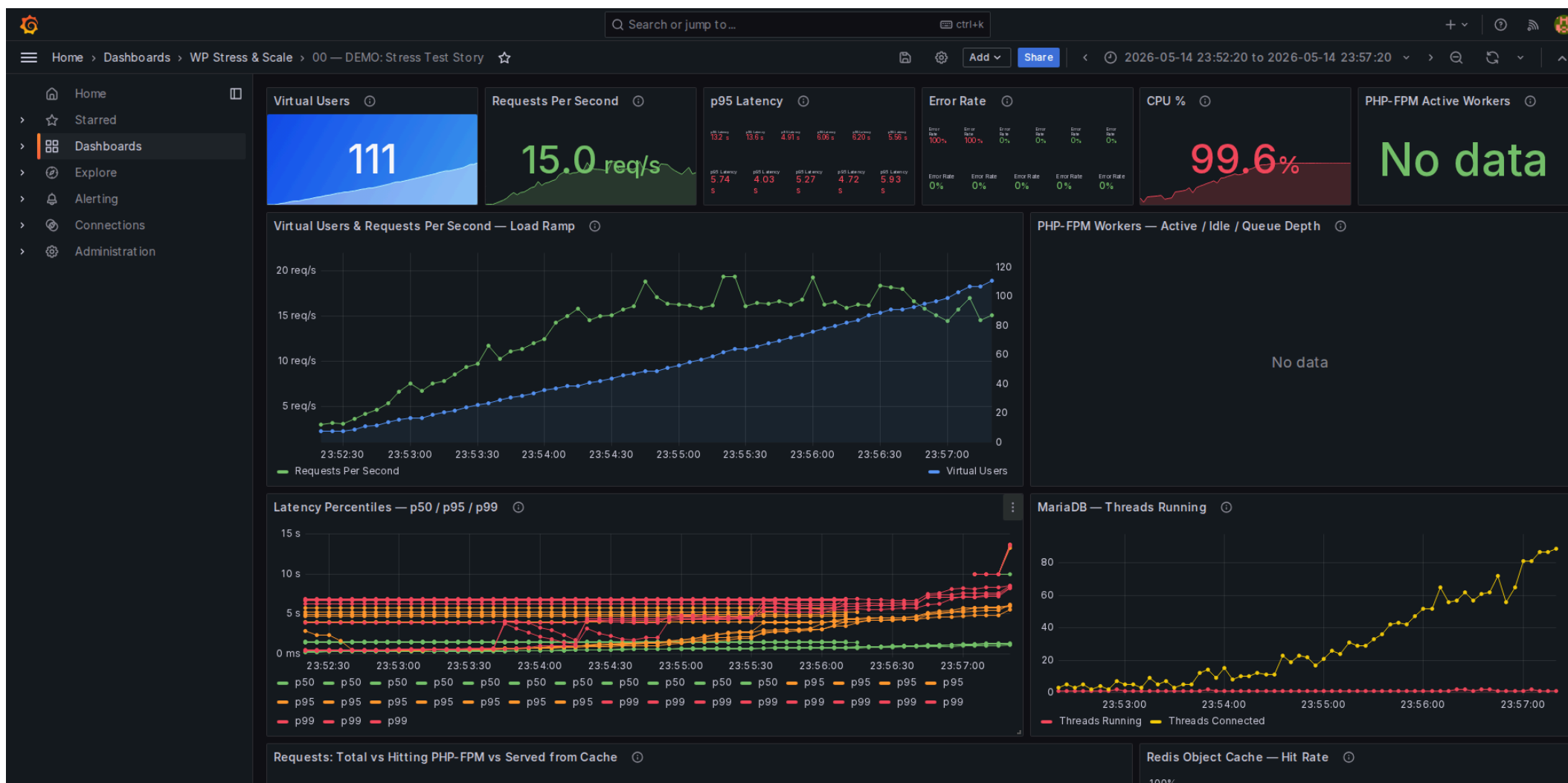
$$\begin{aligned}\text{Max safe workers} &= \frac{\text{RAM} - \text{OS} + \text{DB} + \text{Redis overhead}}{\text{RAM per PHP-WordPress process}} \\ &= \frac{2048 \text{ MB} - 512 \text{ MB}}{32 \text{ MB/worker}} = \mathbf{47 \text{ workers max}}\end{aligned}$$

Workers busy	What happens
1 – 40	✅ Fast responses
41 – 47	⚠️ Slow — swapping begins
48	Queue forms — new requests wait
100+	503 Service Unavailable

Level 0 — Baseline (50 Virtual Users — before the storm)



🔥 Level 0 — The Crash (Black Friday ramp: 0 → 500 Virtual Users)



Level 0 — reading the crash

Panel	Metric	Value	What it means
VU & req/s	Peak requests/s	~140 → collapse	Server buckled — req/s fell while VUs kept climbing
VU & req/s	req/s at 500 VU	24.5	83% of requests failing or queued
Latency	p95	2.9 s	15× worse than Level 1
Latency	p99	6 s flat	Hard ceiling — connections timing out
Latency	p50	~1 s	Even the median is unusable
PHP-FPM	Queue depth	~250	Catastrophic backlog — workers exhausted
PHP-FPM	Active	maxed →	Workers couldn't even start requests



ACT II

Stop the Bleeding

Level 1 — Nginx + PHP-FPM + OPcache + Redis

Level 1 — Three changes, zero cost

1. Nginx (event-driven)

One thread, 10,000+ concurrent connections.

Zero RAM per idle connection.

2. PHP-FPM (decoupled pool)

Workers handle PHP execution only.

Nginx queues connections externally.

`pm.max_children = 20` — hard limit, **never OOM**.

3. OPcache + Redis object cache

- OPcache: compiled PHP bytecode in shared memory — zero disk reads
- Redis: DB query results in RAM — 82% of DB calls served from memory

OPcache — the biggest free win

✗ Without OPcache (Level 0)

Every single request:

```
disk read → parse → lex  
→ AST → compile → opcodes  
→ execute → response
```

Repeat for 300+ PHP files.
wp-settings.php alone
includes 100+ files.

✓ With OPcache (Level 1+)

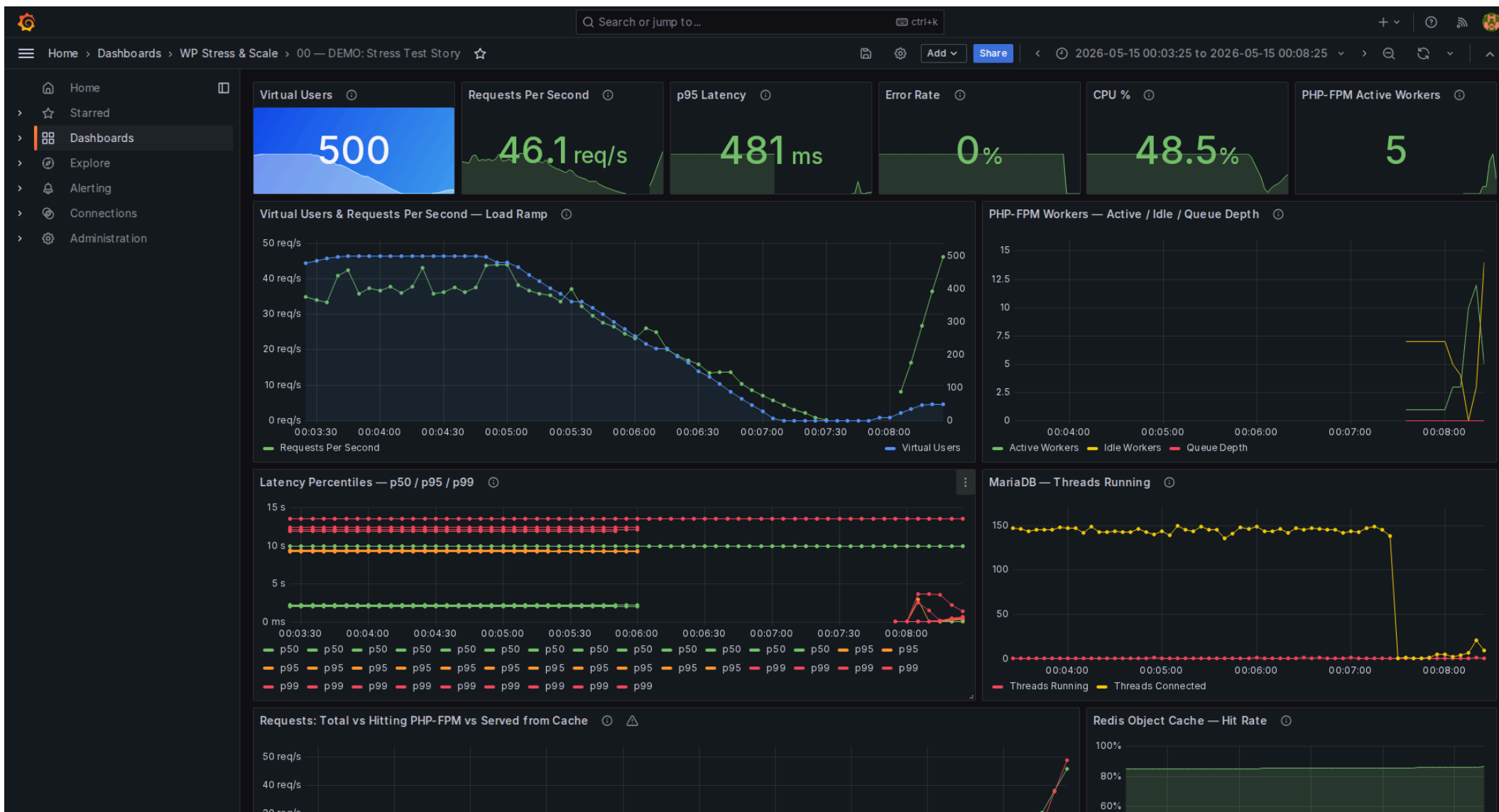
First request only:

```
compile → store in shared RAM
```

Every request after:

```
read opcodes from RAM
```


Level 1 — Demo Dashboard (50 Virtual Users)



Level 1 — reading the dashboard

50

Peak Virtual Users

65

Peak requests/s

133 ms

p95 Latency

FPM Workers chart — top right:

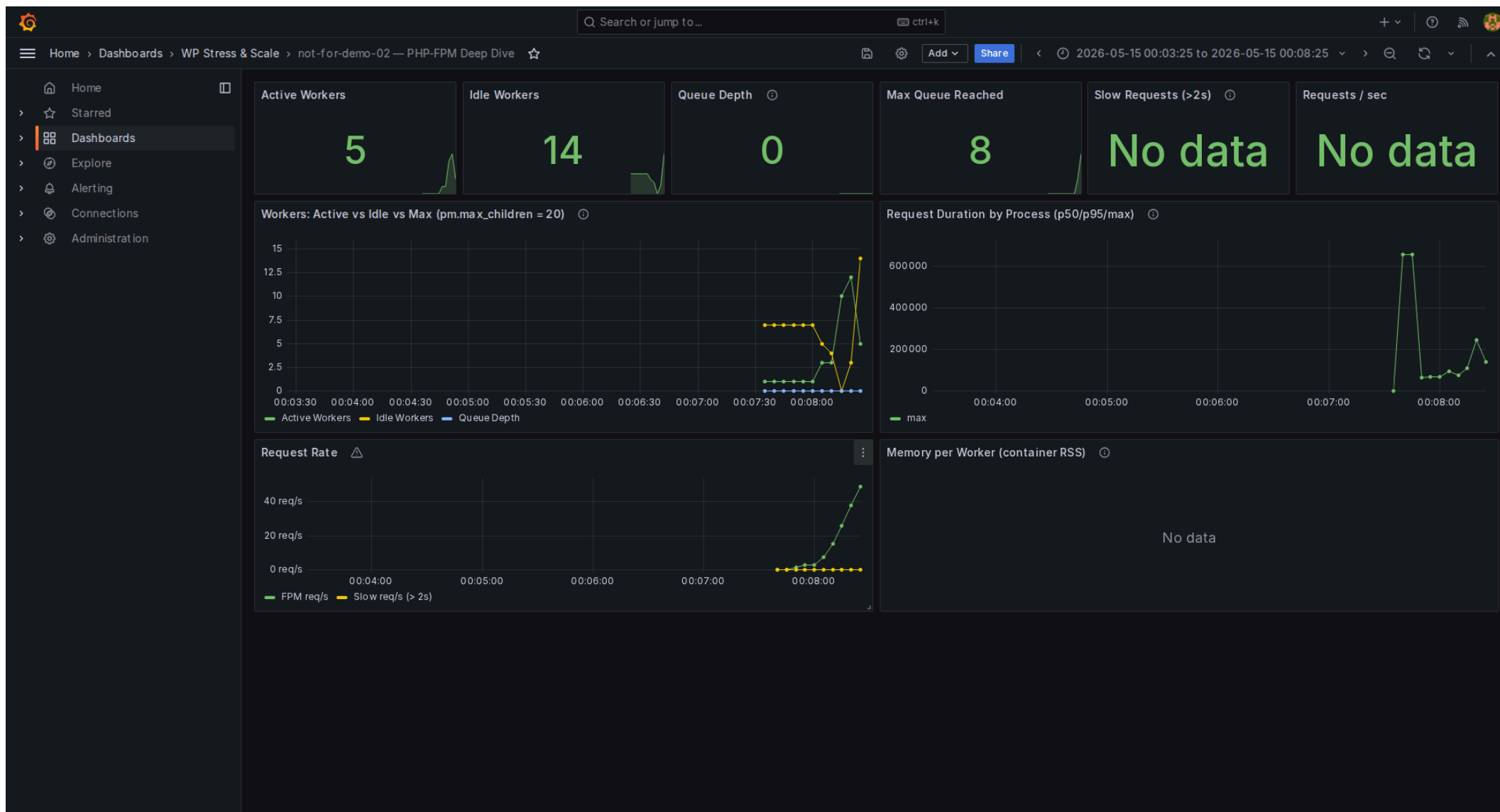
- Active Workers peaked at **14 / 20**
- Queue Depth: **0 throughout** ← the stack did not crash
- Idle workers always available → Nginx never queued

MariaDB Threads — bottom right:

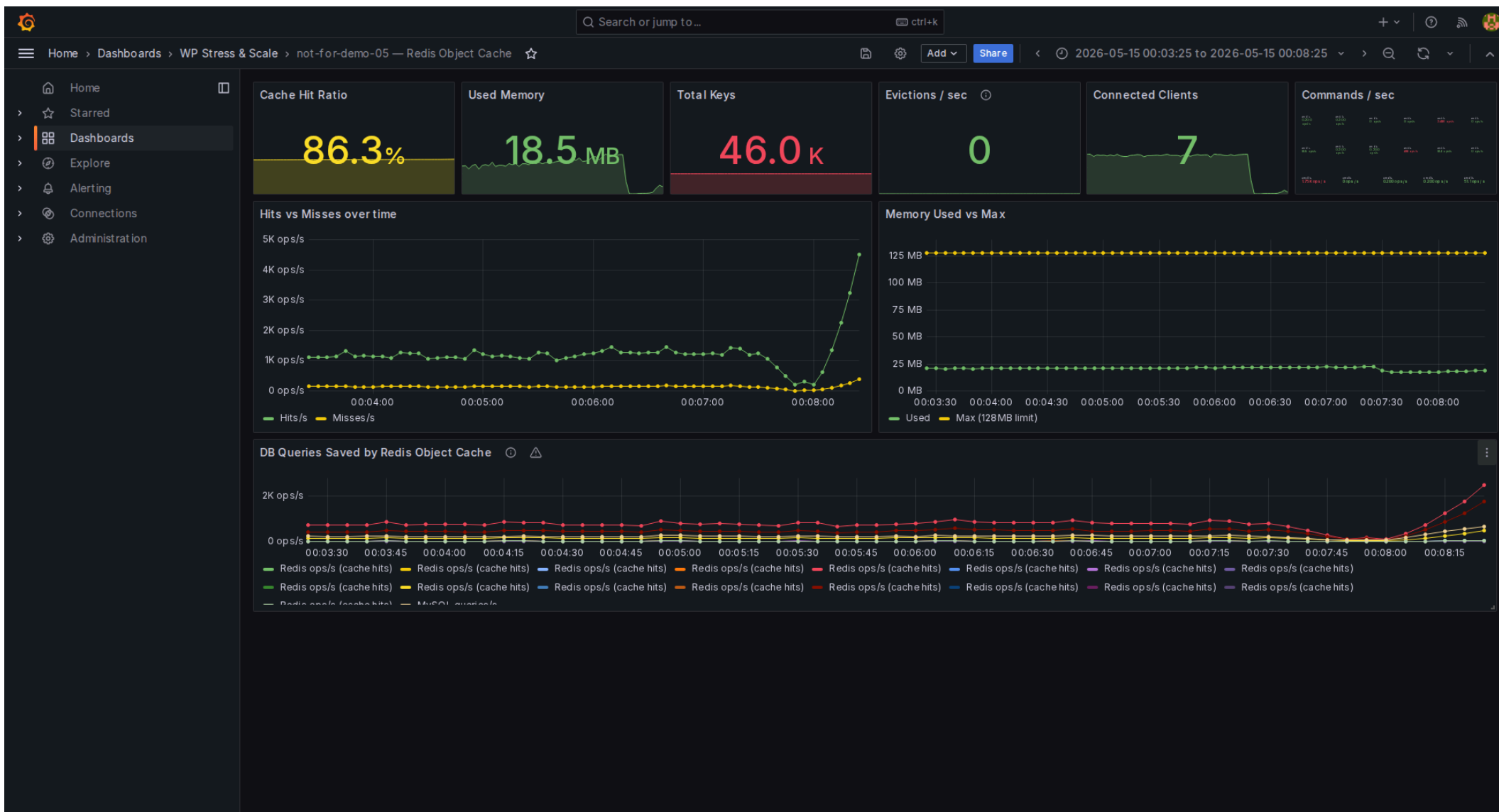
L0 → L1: the jump

Metric	Level 0	Level 1	Δ
Crash at 50 Virtual Users	✗ server buckles	✓ stable	crash eliminated
Peak requests/s	~140 → collapse	65	stable throughput
p95 Latency	~3 s	133 ms	−95%
p99 Latency	6 s flat ceiling	~350 ms	−94%
CPU @ 50 Virtual Users	~100%	25%	−75%
FPM queue depth	~250 backlog	0	✓
DB Threads peak	20+	12.5	−37%
Redis hit ratio	—	82.3%	new layer added

Level 1 — PHP-FPM Workers & Request Rate



Level 1 — Redis: 82% Hit Rate, 6K ops/s



Level 1 — what Redis is actually doing

46,000 cached keys · 14.5 MB · 6,000 ops/s at peak

WordPress makes 300+ DB calls per page request.
Redis intercepts most of them:

<code>wp_get_option('siteurl')</code>	→ Redis HIT → 0.1 ms
<code>wp_get_option('blogname')</code>	→ Redis HIT → 0.1 ms
<code>WP_Query (recent posts)</code>	→ Redis HIT → 0.1 ms
<code>get_post_meta(post_id, '_price')</code>	→ Redis HIT → 0.1 ms
WC product lookup	→ Redis MISS → MySQL → 4 ms → cached

After warmup: ~82% of DB calls answered from memory.
MySQL sees ~54 queries/request instead of 300+.

Redis did not prevent the crash at Level 0 because we never got there.
It **would** delay the crash significantly — but doesn't eliminate it.
That requires removing PHP from the hot path entirely.



ACT III

Cache Everything

Level 2 — FastCGI page cache + MariaDB tuning

The key insight

Most WordPress page requests return identical HTML for every anonymous visitor.

Home page. Category. Blog post. Shop.

PHP runs. MySQL queries. OPcache helps. Redis helps.

The output is still the same HTML every time.

FastCGI page cache — the logical conclusion:

```
First anonymous visitor → PHP runs → nginx stores response on disk
Every visitor after    → nginx reads cached file → 8ms TTFB
                        PHP never runs
                        MySQL never runs
                        Redis never asked
```

Cached (anonymous visitors) 

Homepage · Category pages · Posts · Shop

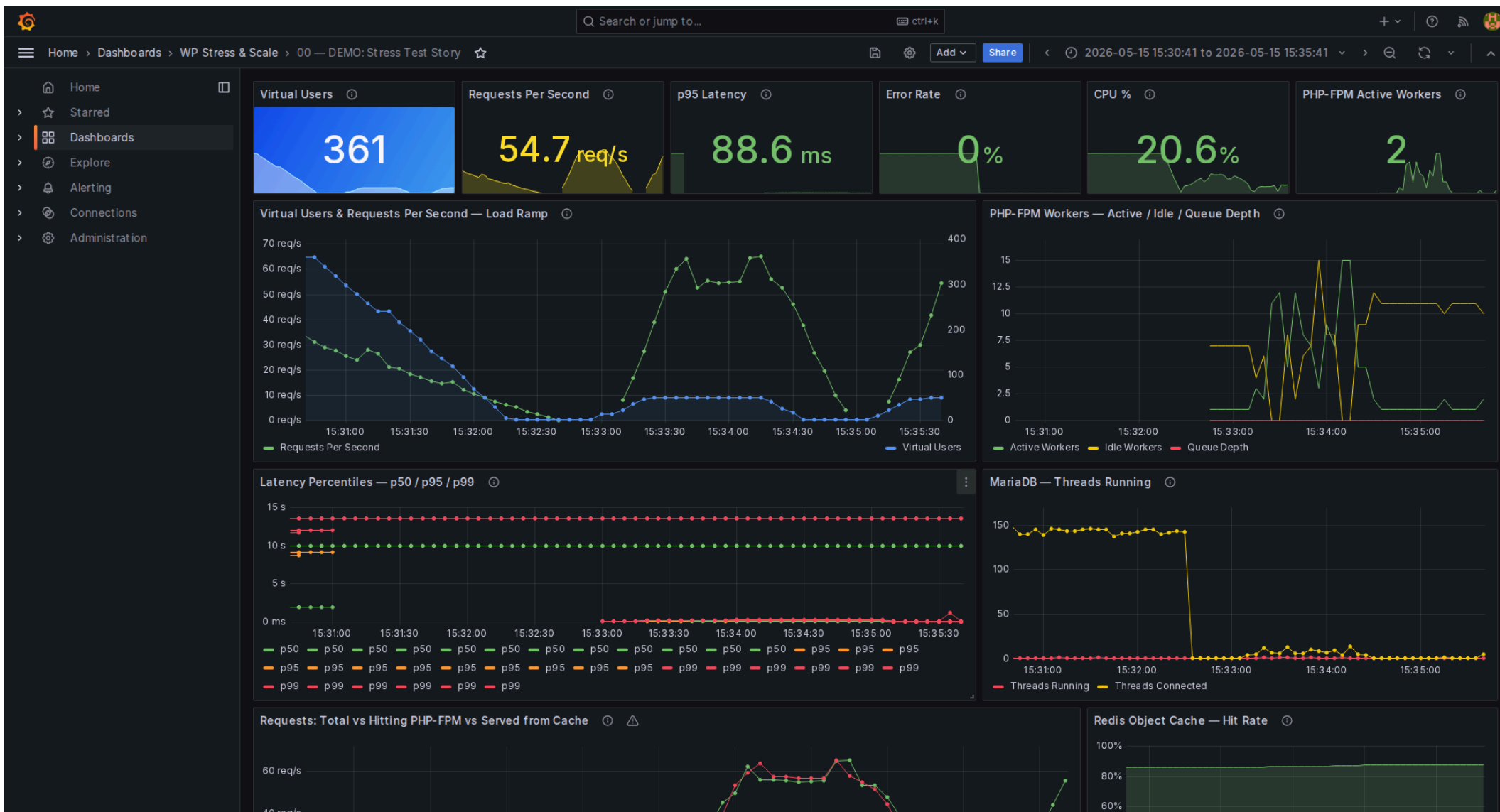
Level 2 — nginx config

```
fastcgi_cache_path /var/cache/nginx
    levels=1:2
    keys_zone=WORDPRESS:100m
    inactive=60m
    max_size=1g;

# Bypass rule — logged-in users and WooCommerce sessions
set $skip_cache 0;
if ($http_cookie ~* "wordpress_logged_in|woocommerce_cart|woocommerce_session") {
    set $skip_cache 1;
}

location ~ /\.php$ {
    fastcgi_cache WORDPRESS;
    fastcgi_cache_key "$scheme$request_method$host$request_uri";
    fastcgi_cache_valid 200 60m;
    fastcgi_cache_bypass $skip_cache;
    fastcgi_no_cache $skip_cache;
    add_header X-Cache-Status $upstream_cache_status; # HIT / MISS / BYPASS
}
```

Level 2 — Demo Dashboard (50 Virtual Users)



Level 2 — reading the dashboard

70

Peak requests/s (+8%)

79 ms

p95 Latency (was 133ms)

18%

CPU (was 25%)

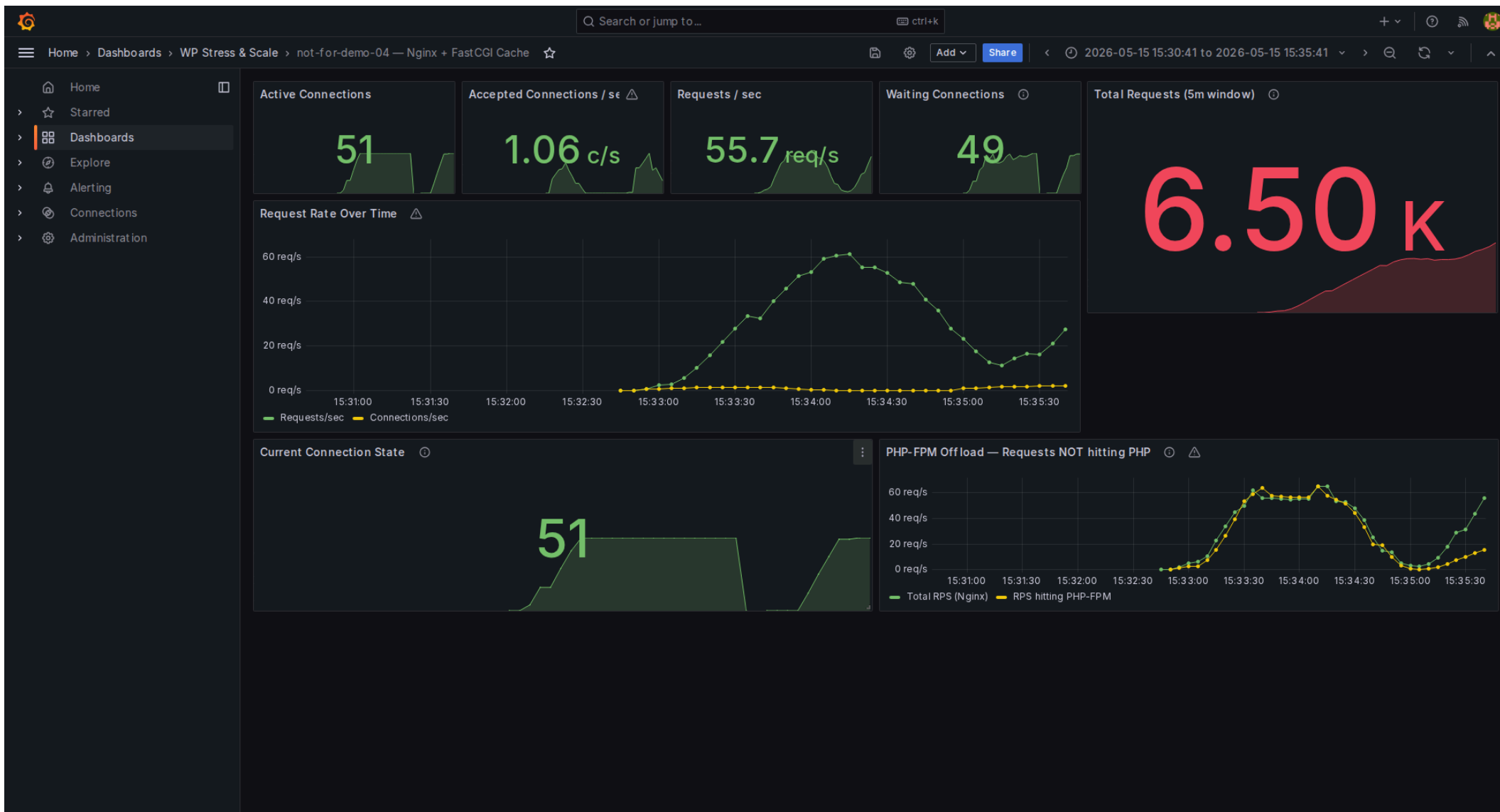
FPM Workers — top right:

- Active Workers peaked at **11 / 20** (was 14)
- Queue Depth: **0** — same as L1 but with less work
- FPM Request Rate: only **~20 requests/s** (vs 70 requests/s from Nginx)
- **50 requests/s served from cache — PHP never ran**

L0 → L1 → L2: the progression

Metric	Level 0	Level 1	Level 2
Peak requests/s	~140 → 	65	70
p95 Latency	~3 s 	133 ms	79 ms
CPU @ 50 Virtual Users	~100%	25%	18%
FPM workers (peak)	—	14 / 20	11 / 20
FPM queue depth	~250	0	0
DB Threads peak	20+	12.5	3
FastCGI cache offload	0%	0%	~70%
Redis hit ratio	—	82.3%	~80%+

🔥 Level 2 — Nginx Cache: 9.94K requests, FPM Offload



Level 2 — the FPM offload panel explained

Bottom-right panel: PHP-FPM Offload — Requests NOT hitting PHP

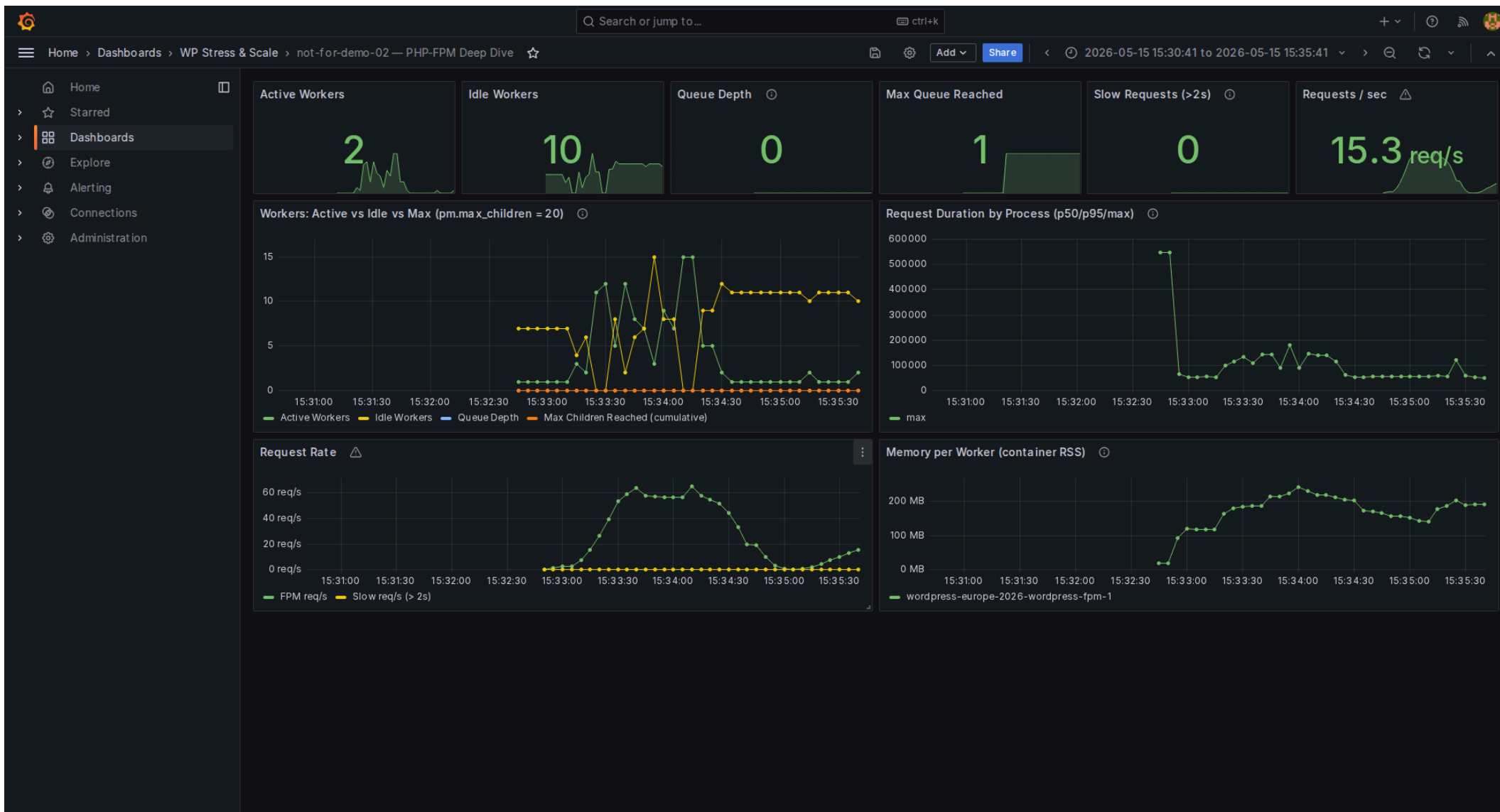


In 5 minutes of the test: 9,940 total requests

- ~7,000 served from nginx cache (disk read, <10ms)
- ~2,940 required PHP execution (cold cache, logged in, dynamic pages)

This is the money slide.

Level 2 — PHP-FPM: Steady at 10-11 Workers



MariaDB tuning — Level 2

Level 0 baseline config:

```
# 00-baseline.cnf
innodb_buffer_pool_size = 128M
# (default – fits almost nothing)
max_connections = 151
```

With 128M buffer pool:

- WordPress DB $\approx$ 500MB
- **Every query reads from disk**
- Buffer pool full $\rightarrow$ constant disk I/O
- Slow queries undetected

Level 2 tuned config:

Level 1 vs Level 2 — the real numbers

Metric	Level 1	Level 2	Change
Peak requests/s @ 50 Virtual Users	65	70	+8%
p95 Latency	133 ms	79 ms	−41%
CPU usage	25.4%	18.1%	−29%
FPM workers (peak)	14 / 20	11 / 20	−21%
FPM queue depth	0	0	—
MariaDB threads (peak)	12.5	3	−76%
Cache offload	0%	~70%	✅ 50 requests/s saved
Requests in 5 min	~8K	9.94K	+24%
Redis hit ratio	82.3%	~80%+	similar



What's Next?

The Static Leap

Level 4 — Simply Static: WordPress as a CMS, not a runtime

Level 4 — the concept

Build time (once per publish cycle, after content is seeded):

```
# Setup – install the Simply Static plugin:  
wp plugin install simply-static --activate  
  
# Export – trigger once after content is ready:  
wp simply-static run  
# Crawls every published URL → flat HTML + assets  
# Output: static/export/ → served by Nginx, zero PHP
```

Simply Static is a WordPress plugin — no external tooling needed.

Run once after seeding 2,500 posts + 500 products. Re-run after any publish.

Takes ~2–5 min. Output directory is the same `static/export/` Nginx already serves.

Runtime (every anonymous request):

Level 4 — Simply Static: expected numbers

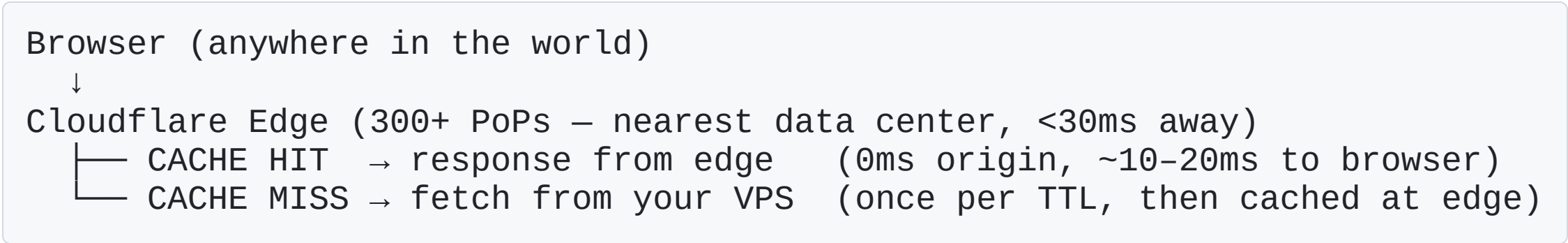
Metric	Level 2	Level 4 (estimated)
requests/s @ 50 Virtual Users	70	500–1,500+
p95 Latency	79 ms	~3–5 ms
CPU @ 50 Virtual Users	18%	<3%
DB Threads	3	~0
PHP workers needed	11	0

The math:

Level 2 bottleneck: PHP executing 300+ DB queries per page
→ Redis saves 82%, FastCGI saves another 70%
→ PHP still runs for cache misses

Level 3 — Cloudflare CDN

The concept:



Your \$12 VPS **only sees cache misses** — ~10–20% of total traffic during a spike.

Metric	Level 2 (direct)	Level 3 (+ Cloudflare)	Δ
TTFB for cached pages	79 ms	10–20 ms	–75%
Origin requests at peak	100%	~15–20%	–80%
Bandwidth from VPS	100%	~20–30%	–75%
DDoS protection	✗ none	✓ unlimited	—

Cloudflare Free Plan — what you get

Performance (free)

-  **Global CDN** — 300+ PoPs worldwide
-  **Full page caching** via Cache Rules
-  **Polish** — automatic image compression
-  **Rocket Loader** — async JS loading
-  **Free SSL/TLS** — automatic HTTPS
-  **HTTP/2 + HTTP/3 (QUIC)** — automatic

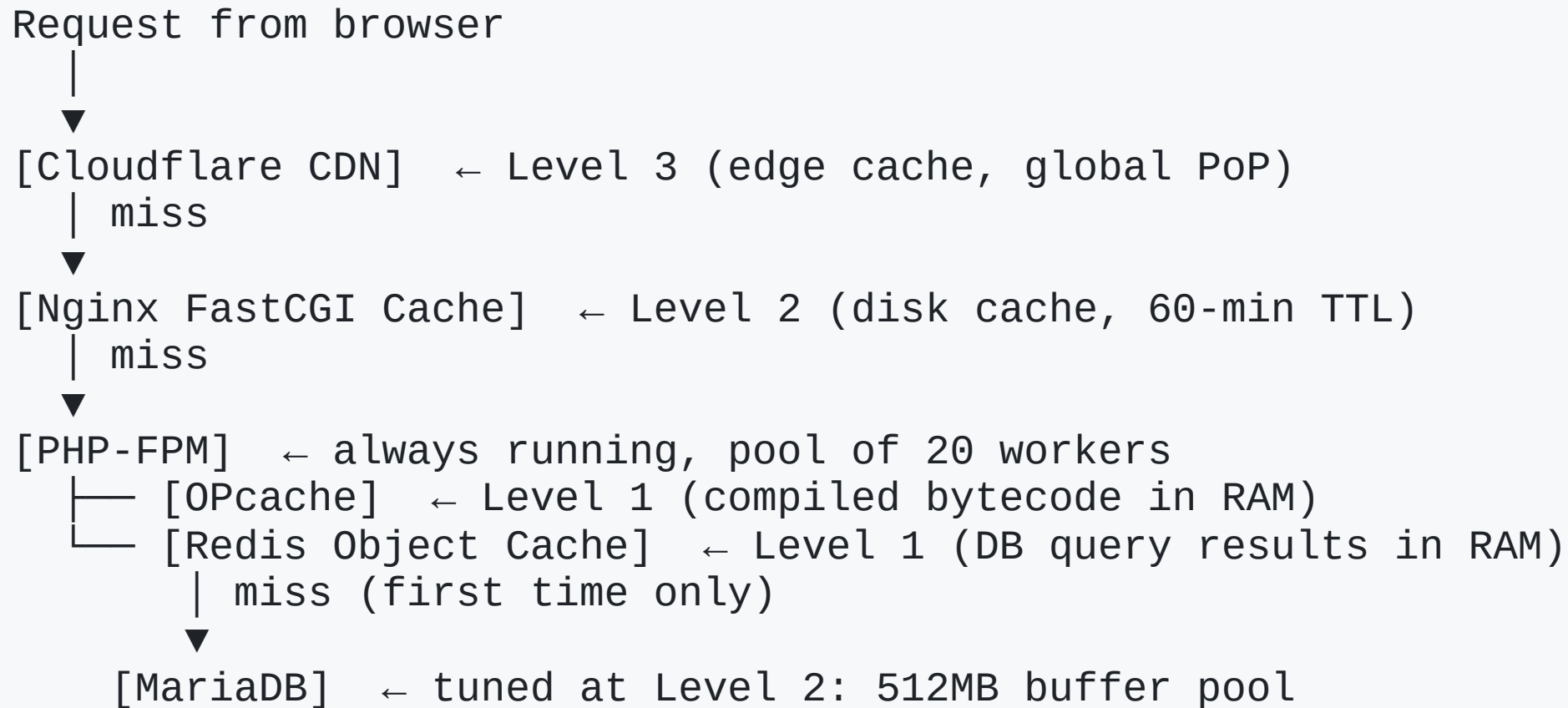
Security (free)

-  **DDoS mitigation** — unlimited, unmetered
-  **Bot Fight Mode** — basic bot blocking
-  **WAF** — 5 custom rules

The full picture

Level	Stack	p95 Latency	CPU @ 50 Virtual Users	DB Threads
0	Apache + mod_php	💣 crash ~50 Virtual Users	100%	20–30
1	Nginx + FPM + OPcache + Redis	133 ms	25%	12.5
2	Level 1 + FastCGI cache + MariaDB	79 ms	18%	3
3	Level 2 + Cloudflare CDN	~20 ms (edge)	<5%	<1
4	Level 2 + Simply Static	~4 ms	<5%	<1

The full cache hierarchy



Each layer absorbed ~70–95% of what reached it.

MariaDB only sees the requests Redis couldn't answer.

Lessons learned

1. Measure before you fix

Set up Prometheus + Grafana

before you need it.

You cannot fix what you cannot see.

```
make obs-up — 30 seconds.
```

2. The three WP cache layers

OPcache	→ PHP bytecode	→ always on
Redis	→ DB queries	→ wp redis enable
FastCGI	→ Full pages	→ nginx.conf

Most sites use 0 of these 3.

All three: completely different machine

The tools — all free, all open source

Tool	Role
k6	Load generator — scriptable, Prometheus remote-write
Prometheus	Metrics collection + storage
Grafana	7 dashboards, auto-provisioned
Loki + Promtail	Container log aggregation

Tool	Role
Nginx FastCGI	Full-page cache (built-in)
PHP OPcache	Bytecode cache (built-in)
Redis	Object cache
MariaDB 11	Tuned DB config

Thank you!

Nedko Hristov

Senior DevOps Engineer @ Nemetschek Bulgaria



`github.com/NedkoHristov/WordPress-Europe-2026`

Questions?

*"The best time to add monitoring was before the crash.
The second best time is right now."*